PokeService
- singleton: PokeService
- library: HashMap<PokeEnum, PokemonInfo>

+ getSingleton(): PokeService
+ getAvailablePokemons(): List<PokemonInfo>
+ getInfo(PokeEnum id): PokemonInfo
+ getStarters(): List<PokemonInfo>
- PokeService()
- register()
- registerInfo(PokeEnum id, PokemonInfo info)

1 — Pokemon registrati (0..*) — Ottiene

<<Enum>> PokeEnum
NONE
BULBASAUR
IVYSAUR
VENUSAUR
CHARMANDER
...
MEW = 151

<<Interface>> PokemonInterface
+ getName(): String
+ getId(): PokeEnum
+ getElement(): PokeElement
+ getLevelType(): LevelUpGroup
+ getCurrentEvolution(): int
+ getMaxEvolution(): int
+ getDropExp(): int
+ hasEvolution(): boolean
+ evolveAtLevel(): int

<<Enum>> PokeElement
NORMAL
FIRE
...
DRAGON = 14

<<Interface>> MoveInterface
+ getName(): String
+ getElement(): PokeElement
+ getType(): MoveType
+ getMaxUses(): int
+ getPower(): int
+ getAccuracy(): int
+ getRequiredLevel(): int
+ perform(Pokemon attacker, Pokemon defender): MoveResult

<<Enum>> MoveType
MISS
PHYSICAL
SPECIAL
STATUS

MoveResult
- damage: int
- moveResult: AttackEffectiveness
- elemResult: AttackEffectiveness
- critical: boolean
- effects: PokeStats
- tpye: MoveType

+ MoveResult(int, boolean, AttackEffectiveness, AttackEffectiveness)
+ MoveResult(PokeStats effects)
+ MoveResult()
+ getType(): MoveType
+ getEffects(): PokeStats
+ getDamange(): int
+ getMoveResult(): AttackEffectiveness
+ getElemResult(): AttackEffectiveness
+ isCritical(): boolean

2 — **<<Enum>> AttackEffectiveness**
MISS
NOT_EFFECTIVE
NORMAL
EFFECTIVE;

+ getText(): String
+ getEffectiveness(): double

Pokemon
- info: PokemonInfo
- moves: PokeMove[4]
- health: int
- iv: PokeStats
- ev: PokeStats
- effects: PokeStats
- level: int
- exp: int

+ Pokemon(PokemonInfo)
+ getInfo(): PokemonInfo
+ isDead(): boolean
+ canEvolve(): boolean
+ evolve()
+ getExpToNextLevel(): int
+ canLevelUp(): boolean
+ levelUp()
+ gainExp(Pokemon, int)
+ getMoves(): List<PokeMove>
+ hasMove(BaseMove) boolean
+ canLearnMove(): PokeMove
+ addMove(PokeMove)
+ getHealth()
+ restoreHealth()
+ restore()
+ setLevel()
+ getLevel(): int
+ setPs(int)
+ getTotPs(): int
+ setSpeed(int)
+ getTotSpeed(): int
+ setAttack(int)
+ getTotAttack(): int
+ setDefense(int)
+ getTotDefense(): int
+ setSpecial(int)
+ getTotSpecial(): int
+ getTotStats(): PokeStats
+ heal(int)
+ damage(int)
- calculateStat(int base, int iv, int pa): int

Detiene Mosse (0..4) — **PokeMove**
- move: BaseMove
- usesLeft: int
- slot: int

+ PokeMove(BaseMove)
+ getSlot(): int
+ setSlot(int)
+ getUsesLeft(): int
+ consumeUses()
+ resetUses()
+ canUse(): boolean

1 — Informazioni base della mossa — ***BaseMove***
- requiredLevel: int

+ BaseMove(int)
+ getRequiredLevel(): int
+ getEffects(): PokeStats
+ calculateDamage(Pokemon attacker, Pokemon defender): MoveResult

0..* — Mosse registrate — Use

Implementazioni: Growl, Tackle, Ember, TailWhip, VineWhip, WaterGun, Scratch, DragonBreath, DragonClaw

Informazioni base (1) — ***PokemonInfo***
baseStats: PokeStats

+ getRequiredExp(int level): int
+ getNextEvolution(): PokemoninfoInterface

Implementazioni: Squirtle, Wartortle, Blastoise

Statistiche base (1) — Statistiche personali (3)

<<Interface>> StatsInterface
+ getPS(): int
+ getAttack(): int
+ getDefense(): int
+ getSpecial(): int
+ getSpeed(): int

PokeStats
- attack: int
- defense: int
- special: int
- speed: int
- ps: int
- maxValue: int

+ PokeStats(int maxValue)
+ PokeStats()

0..1 — Mossa dell'azione — Il pokemon che si vuole schierare (1)

0..6 — Possiede dei Pokemon

Trainer
- pokemons: Pokemons[6]
- name: String
- currentPokemon: int

+ getTeamSize(): int
+ getPokemons(): Pokemon[]
+ getPokemon(int slot): Pokemon
+ setPokemon(int index, Pokemon pokemon)
+ addPokemon(Pokemon)
+ getActivePokemon(): Pokemon
+ setActivePokemon(int index)
+ setActivePokemon(Pokemon)
+ getAvailablePokemons(): List<Pokemon>
+ getAllPokemons(): List<Pokemon>
+ getFirstAvailablePokemon(): Pokemon
+ getTeamLevel(): int
+ getName(): String
+ setName(String)

Actors (0..2) — **BattleAction**
- type: BattleActionType
- trainer: Trainer
- opponent: Trainer
- pokemon: Pokemon
- move: PokeMove

+ BattleAction(Trainer, Trainer, PokeMove)
+ BattleAction(Trainer, Pokemon pokemon)
+ BattleAction()
+ getType(): BattleActionType
+ getTrainer(): Trainer
+ getOpponent(): Trainer
+ getPokemon(): Pokemon
+ getMove(): PokeMove
+ performAction(): MoveResult?
- BattleAction(Trainer trainer, Trainer opponent)
- applyAttack(MoveResult attack)

L'azione che si sta eseguendo (2) — **<<Enum>> BattleActionType**
SKIP
ATTACK
POKEMON_SWITCH

MoveService
- singleton: MoveService
- library: HashMap<PokeEnum, ArrayList<BaseMove>>

+ getSingleton(): MoveService
+ getMoves(PokeEnum): List<BaseMove>
+ getMovesAtLevel(PokeEnum id, int level): BaseMove?
+ getSupportedMoves(Pokemon): List<BaseMove>
- MoveService()
- registerMoves()
- registerMove(PokeEnum id, BaseMove move)

Use (1)

GameState
- singleton: GameState
- player: Trainer
- score: int

+ getSingleton(): GameState
+ increaseScore()
+ getScore(): int
+ resetScore()
+ setPlayer(Trainer)
+ getPlayer(): Trainer
+ getCurrentDifficulty(): Difficulty

return — **<<Enum>> Difficulty**
EASY
MEDIUM
HARD
IMPOSSIBLE
HAHAHAHAH

Use — **PokeBuilder**
- random: Random
- minLevel: int
- maxLevel: int
- maxPS: int
- maxAttack: int
- maxDefense: int
- maxSpecial: int
- maxSpeed: int
- id: PokeEnum

+ PokeFactory(PokeEnum)
+ PokeFactory()
+ setId(PokeEnum)
+ setMinLevel(int)
+ setMaxLevel(int)
+ generate(): Pokemon
- generateMoves(Pokemon)

Use — **TrainerBuilder**
- pokeFactory: PokeFactory
- randomizer: PokeRandomizer
- random: Random
- generateTeam: boolean
- useStarters: boolean
- minTeamSize: int
- maxTeamSize: int
- NAMES: String[]

+ generate(): Trainer
- generateName(Trainer)
- generateTeam(Trainer)
- getRandomName(): String
+ setMinTeamSize(int)
+ setMaxTeamSize(int)
+ setMinPokeLevel(int)
+ setMaxPokeLevel(int)
+ setMinEvolution(int)
+ setMaxEvolution(int)
+ useStarters(boolean)

Generazione Team (1)

PokeRandomizer
- service: PokeService
- random: Random
- minEvolution: int
- maxEvolution: int

+ setMaxEvolution(int)
+ setMinEvolution(int)
+ getAvailablePokemons(): List<PokemonInfo>
+ getRandomInfo(int size): PokemonInfo[]
+ getRandomUniqueInfo(int size) PokemonInfo[]
+ getRandomStarters(size): List<PokemonInfo>

Selettore di Pokemon (1)

Azioni battaglia ordinate per priorita' (1)

TurnHandler
- player1: Trainer
- player2: Trainer
- player1Action: BattleAction
- player2Action: BattleAction

+ setPlayer1Action(BattleAction)
+ setPlayer2Action(BattleAction)
getOrderedActions(): List<BattleAction>

<<Interface>> CPUInterface
+ handlePokemonChange(): Pokemon
+ handleTurn: BattleAction

CPUHandler
cpu: Trainer
opponent: Trainer
random: Random

+ CPUHandler(Trainer cpu, Trainer opponent)
makeAction(Pokemon): BattleAction
makeAction(PokeMove): BattleAction

EasyCPUHandler — MediumCPUHandler

BattleHandler
- state: GameState
- cpuHandler: CPUHandler

+ BattleHandler(GameState)
+ getCPU(): Trainer
+ generateNewOpponent()
+ startTurn(): Queue<BattleAction>
+ playerCatchPokemon(): Pokemon
- setupCPU()

Leaderboard
- singleton: Leaderboard
- scores: ArrayList<PlayerScore>

+ getSingleton(): Leaderboard
+ getMinScore(): int
+ getScores(): List<PlayerScore>
+ addScore(String, int)
+ addScore(PlayerScore)
+ save()

1 — Lista di punteggi registrati (0..10) — **PlayerScore**
- name: String
- score: int

+ PlayerScore(String, int)
+ getName(): String
+ getScore(): int

Use

Use — **SoundService**
- singleton: SoundService
- effects: ArrayList<AudioClip>
- musics: ArrayList<AudioClip>

+ getSingleton(): SoundService
- loadAudio(String name, SoundType)
+ playPurchase()
+ playPress()
+ getTitleAudio(): AudioClip
+ getBattleAudio(): AudioClip
+ getVictoryAudio(): AudioClip

Use — **Controllers**

Application — User

JPokeBattle
+ WIDTH: int
+ HEIGHT: int
- instance: JPokeBattle
- stage: Stage

+ getInstance(): JPokeBattle
+ setView(String)
+ getScene(): Scene
+ getStage(): Stage

Mostra — StartController — Primo Input — MenuController

LeaderController (Indietro / Classifica)
FinishController (Menu / Fine)
MenuController: Nuova Partita → SetupController (Inizio → BattleController); Indietro; Esci/Fuga[score==0]
InfoController (Indietro / Informazioni)

BattleController
- game: GameState
- battleHandler: BattleHandler
- actions: Queue<BattleAction>
- lastAction: BattleAction
- freeSwitch: boolean
- player: Trainer
- catchingPokemon: Pokemon
- learningMove: PokeMove

+ runAway()
- startTurn(BattleAction playerAction)
- handleAction(BattleAction action)
...

Use — GUI

ArrowTextButton
- button: Button
- arrow: ImageView

+ ArrowTextButton()
+ ArrowTextButton(String text, boolean enabled)
+ removeArrow()
+ addArrow()
+ getButton(): Button
+ setText(String)
+ showArrow(boolean)
+ resizeArrow(int width, int height)

ArrowButton
- arrow: Image
- imageView: ImageView

+ ArrowButton(int width, int height)
+ ArrowButton()
+ resize(int, int)

GridMenu
- options: MenuOption[]
- selected: MenuOption
- clickCallback: MenuCallback
- selectCallback: MenuCallback

+ GridMenu()
+ update(MenuOption[] options, int cols)
+ selectOption(MenuOption)
+ setArrowSize(int, int)
+ setClickCallback(MenuCallback)
+ setSelectCallback(MenuCallback)
- handleClick(MenuOption option)
- handleMouseHover(MenuOption option)

0..* — **MenuOption**
- text:String
- enabled: boolean
- value: int
button: ArrowTextButton

+ getText(): String
+ setText(String)
+ isEnabled(): boolean
+ setEnabled(boolean)
+ getValue(): int
+ setValue(int)
+ showArrow(boolean show)

HealthBar
- healthRect: Rectangle
- tweenTimeline: TimeLine

+ HealthBar()
- calculateHealthWidth(Pokemon)
- getColorFromHealth(Pokemon)
+ updateColor(Pokemon)
+ tweenHealth(Pokemon, PokeCallback)
- stopAnimation()
+ updateHealth(Pokemon)

HPokeBalls

+ HPokeBalls()
- getPokeBalls(): ImageView
+ update(Trainer)

LeaderboardGUI
- tableView: TableView<PlayerScore>

+ LeaderboardGUI()
+ update(List<PlayerScore>)

PokeCard
- pokemonView: PokemonView
- pokemonName: Label
- levelLabel: Label
- psLabel: Label
- arrow: ImageView
- healthBar: HealthBar

+ PokeCard()
+ toggleArrow(boolean)
+ update(Pokemon)

PokeInfoCard
- nameLabel: Label
- hpLabel: Label
- levelLabel: Label
- pokemon: Pokemon
- healthBar: HealthBar

+ PokeInfoCard()
+ update(Pokemon)
+ tweenHealth(PokeCallback onFinished)
+ updateHealth()
+ removeHpLabel()
+ getHealthBar(): HealthBar
- updateHealthLabel()

PokemonView
- id: PokeEnum
- renderBack: boolean

+ updateImage()
+ setPokemon(PokeEnum)
+ setRenderBack(boolean)

<<Interface>> ClickCallback
+ onClick(int, Pokemon)

Use — **PokeTeamList**
- currentCard: PokeCard
- callback: ClickCallback

+ PokeTeamList()
+ update(Trainer)
+ setCallback(ClickCallback)
- toggleCurrentCardArrow()